

An Analysis of Softmax function

Softmax function

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

Softmax function -- Part 1

$\sigma(z)_i = \frac{e^{\beta z_i}}{\sum_{j=1}^K e^{\beta z_j}}$ or $\sigma(z)_i = \frac{e^{-\beta z_i}}{\sum_{j=1}^K e^{-\beta z_j}}$ for $i = 1, \dots, K$.
 $\sigma(z)_i = \frac{e^{\beta z_i}}{\sum_{j=1}^K e^{\beta z_j}}$ or $\sigma(z)_i = \frac{e^{-\beta z_i}}{\sum_{j=1}^K e^{-\beta z_j}}$ for $i = 1, \dots, K$.

Softmax function -- Part 2

For any input, the outputs must all be positive and they must sum to unity. ... Given a set of unconstrained values, $V_j(x)$, we can ensure both conditions by using a Normalised Exponential transformation:

Softmax function -- Part 3

Applications The softmax function is used in various multiclass classification methods, such as multinomial logistic regression (also known as softmax regression), multiclass linear discriminant analysis, naive Bayes classifiers, and artificial neural networks. Specifically, in multinomial logistic regression and linear discriminant analysis, the input to the function is the result of K distinct linear functions, and the predicted probability for the j th class given a sample tuple x and a weighting vector w is:

Softmax function -- Part 4

Alternatives The softmax function generates probability predictions densely distributed over its support. Other functions like sparsemax or α -entmax can be used when sparse probability predictions are desired. Also the Gumbel-softmax reparametrization trick can be used when sampling from a discrete-discrete distribution needs to be mimicked in a differentiable manner.

Softmax function -- Part 5

In words, the softmax applies the standard exponential function to each element z_i of the input tuple z (consisting of K real numbers), and normalizes these values by dividing by the sum of all these exponentials. The normalization ensures that the sum of the components of the output vector $\sigma(z)$ is 1. The term "softmax" derives from the

amplifying effects of the exponential on any maxima in the input tuple. For example, the standard softmax of $(1, 2, 8)$ is approximately $(0.001, 0.002, 0.997)$, which amounts to assigning almost all of the total unit weight in the result to the position of the tuple's maximal element (of 8). In general, instead of e a different base $b > 0$ can be used. As above, if $b > 1$ then larger input components will result in larger output probabilities, and increasing the value of b will create probability distributions that are more concentrated around the positions of the largest input values. Conversely, if $0 < b < 1$ then smaller input components will result in larger output probabilities, and decreasing the value of b will create probability distributions that are more concentrated around the positions of the smallest input values. Writing $b = e^{\beta}$ or $b = e^{-\beta}$ (for real β) yields the expressions:

Softmax function -- Part 6

Here, the Kronecker delta is used for simplicity (cf. the derivative of a sigmoid function, being expressed via the function itself). To ensure stable numerical computations subtracting the maximum value from the input tuple is common. This approach, while not altering the output or the derivative theoretically, enhances stability by directly controlling the maximum exponent value computed. If the function is scaled with the parameter β , then these expressions must be multiplied by β . See multinomial logit for a probability model which uses the softmax activation function.

Softmax function -- Part 7

Neural networks The standard softmax function is often used in the final layer of a neural network-based classifier. Such networks are commonly trained under a log loss (or cross-entropy) regime, giving a non-linear variant of multinomial logistic regression. Since the function maps a tuple and a specific index i to a real value, the derivative needs to take the index into account:

Softmax function -- Part 8

$$e^{m_i - 1 - m_i} \cdot i - 1 + e^{z_i - m_i} \cdot i = e^{z_i - 1 - m_i} \cdot i + e^{z_i - m_i} \cdot i - m_i \cdot i =$$

Softmax function -- Part 9

$$P_t(a) = \exp\left(\frac{q_t(a)}{\tau}\right) \sum_{i=1}^n \exp\left(\frac{q_t(i)}{\tau}\right), \\ P_{t+1}(a) = \frac{\exp(q_{t+1}(a)/\tau)}{\sum_{i=1}^n \exp(q_{t+1}(i)/\tau)}$$

An Analysis of Softmax function

Softmax function

Softmax function -- Part 10

However, if the difference is small relative to the temperature, the value is not close to the arg max. For example, a difference of 10 is small relative to a temperature of 100:

Softmax function -- Part 11

$$\sigma(\mathbf{z} + \mathbf{c})_j = \frac{e^{z_j + c}}{\sum_{k=1}^K e^{z_k + c}} = e^{z_j} \cdot e^c \cdot \frac{1}{\sum_{k=1}^K e^{z_k + c}} = \sigma(\mathbf{z})_j \cdot e^c$$

Softmax function -- Part 12

As $\beta \rightarrow \infty$, temperature goes to zero, $T = 1 / \beta \rightarrow 0$, so eventually all differences become large (relative to a shrinking temperature), which gives another interpretation for the limit behavior.

Softmax function -- Part 13

History The softmax function was used in statistical mechanics as the Boltzmann distribution in the foundational paper Boltzmann (1868), formalized and popularized in the influential textbook Gibbs (1902). The use of the softmax in decision theory is credited to R. Duncan Luce, who used the axiom of independence of irrelevant alternatives in rational choice theory to deduce the softmax in Luce's choice axiom for relative preferences. In machine learning, the term "softmax" is credited to John S. Bridle in two 1989 conference papers, Bridle (1990a); and Bridle (1990b):

Softmax function -- Part 14

$$\sigma(0, 10) := \sigma(0, 10) = (1 / (1 + e^{10}), e^{10} / (1 + e^{10})) \approx (0.00005, 0.99995)$$